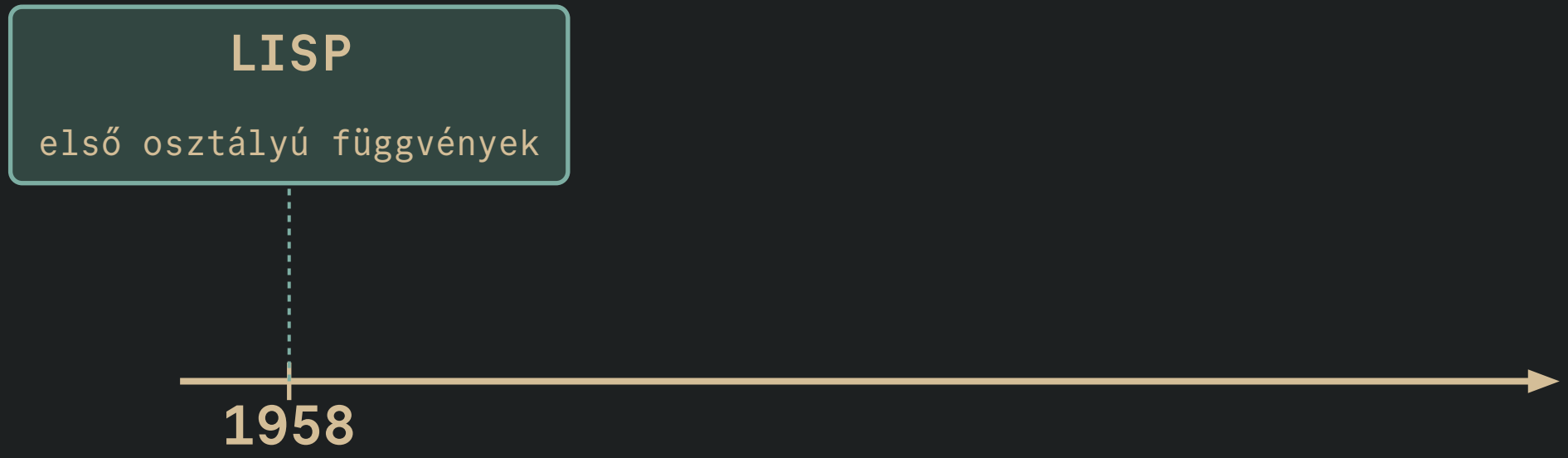
Combinator Explorer

Orbán Levente

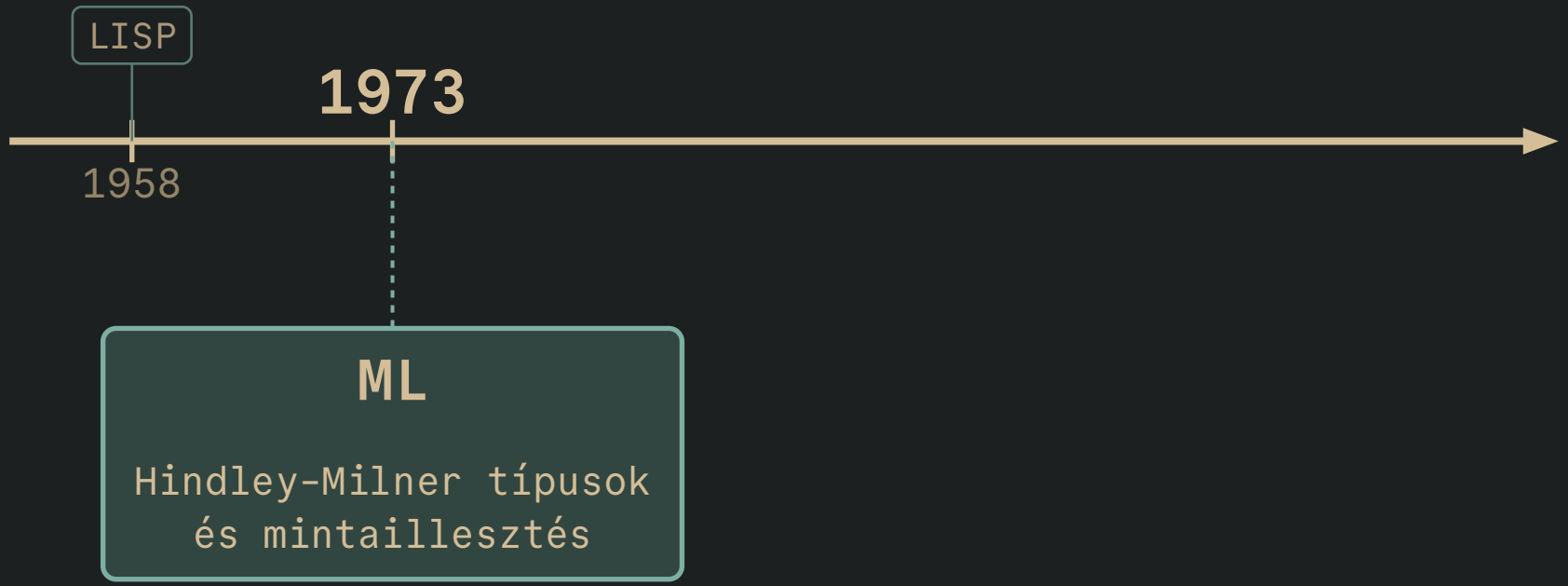
Konzulens: Németh Gábor

2026-05-27

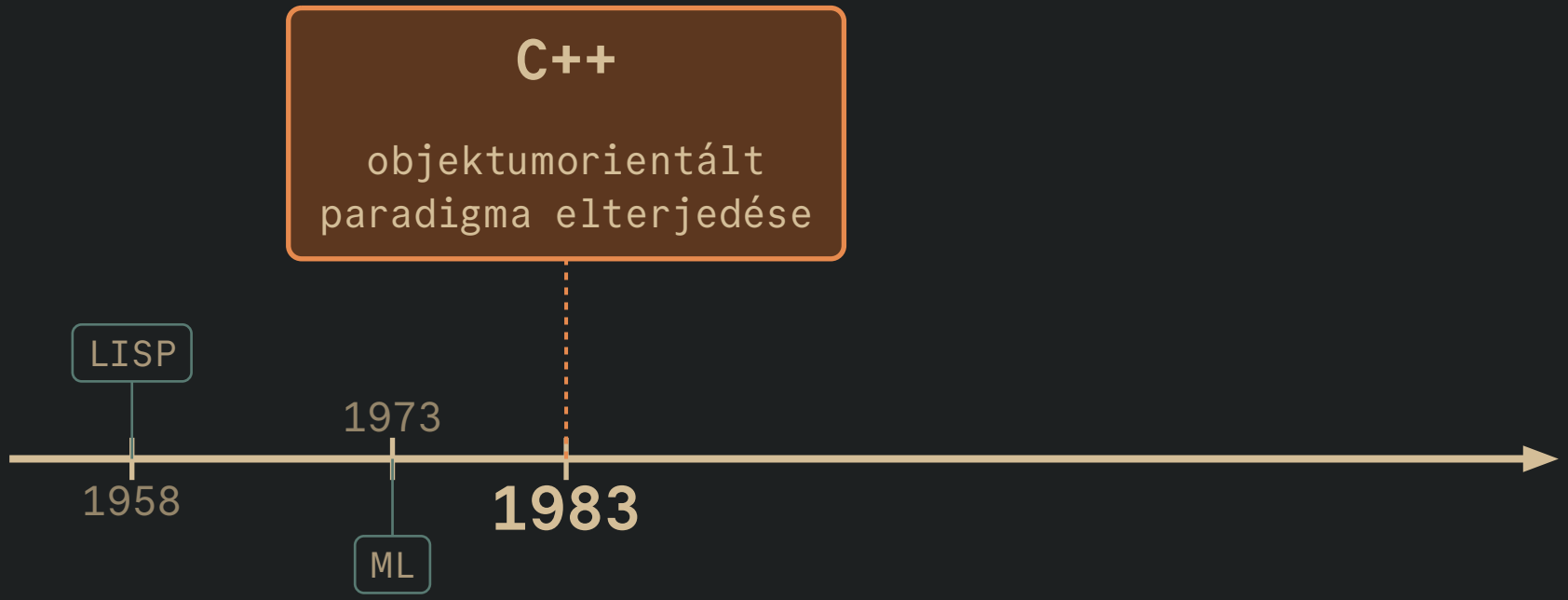
paradigmaváltozás



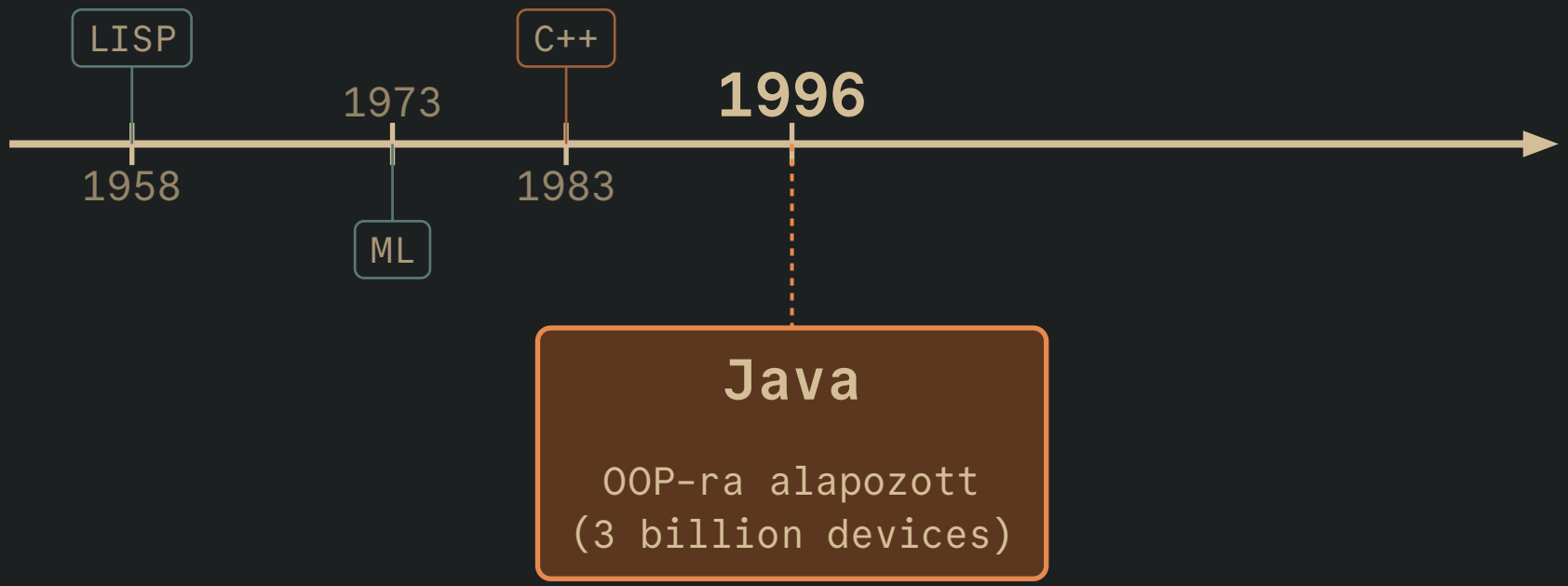
paradigmaváltozás



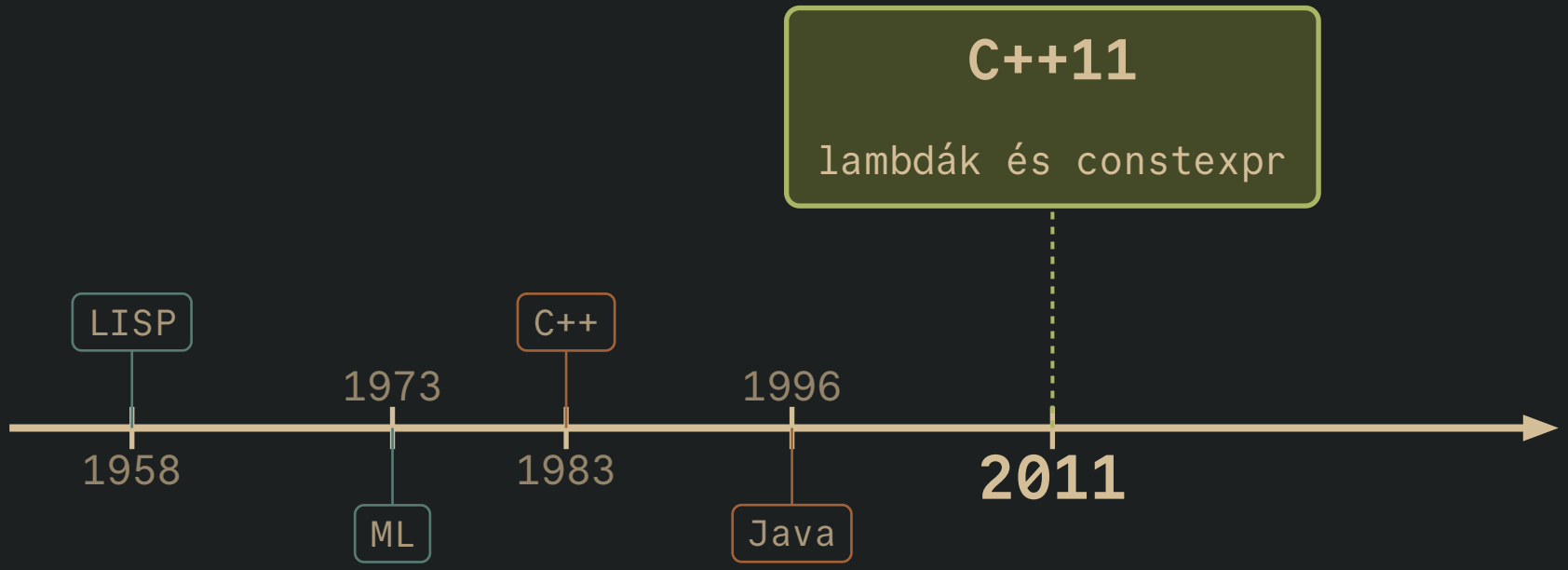
paradigmaváltozás



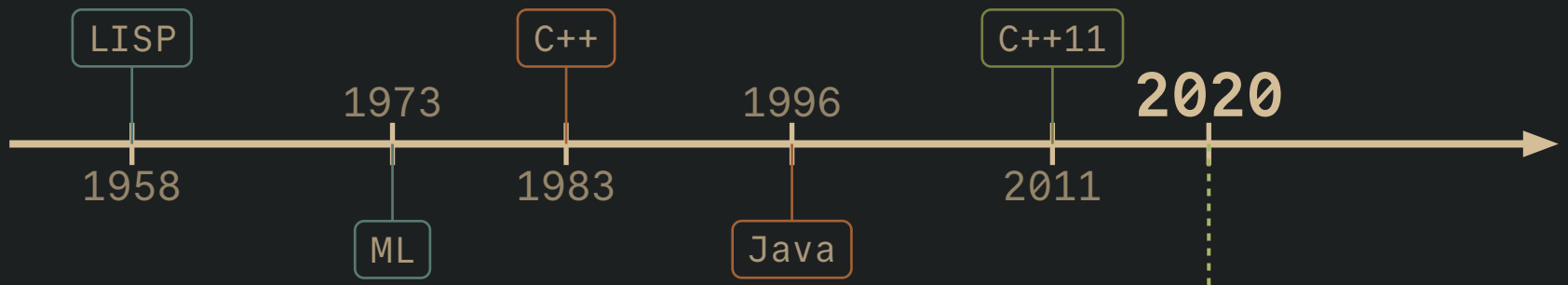
paradigmaváltozás



paradigmaváltozás



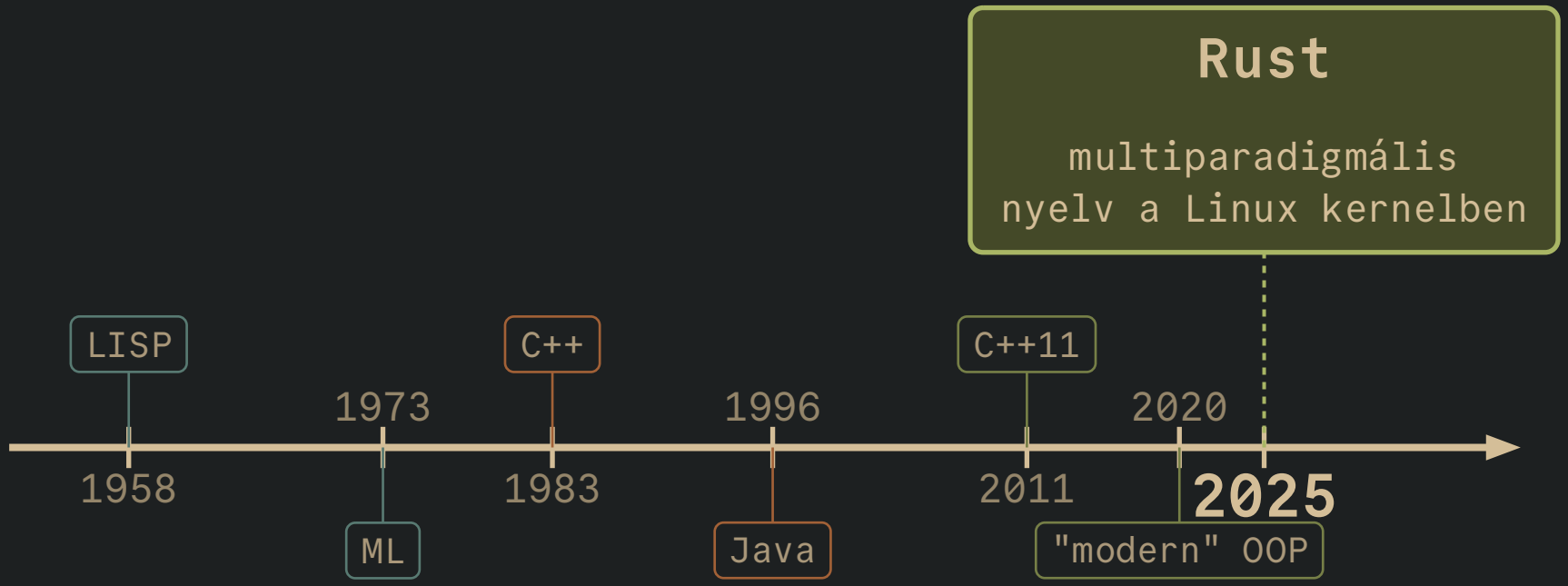
paradigmaváltozás



"modern" OOP

rekordok, mintaillesztés
(Java, C#, Kotlin)

paradigmaváltozás



paradigmaváltozás

- az újabb nyelvek már **multiparadigmálisak** (Rust, Kotlin)

paradigmaváltozás

- az újabb nyelvek már **multiparadigmálisak** (Rust, Kotlin)
- a régebbi nyelvek is efelé mozognak

paradigmaváltozás

- az újabb nyelvek már **multiparadigmálisak** (Rust, Kotlin)
- a régebbi nyelvek is efelé mozognak
- az eszközök (lambdák, mintaillesztés) mindennaposak lettek

paradigmaváltozás

- az újabb nyelvek már **multiparadigmálisak** (Rust, Kotlin)
- a régebbi nyelvek is efelé mozognak
- az eszközök (lambdák, mintaillesztés) mindennaposak lettek
- a mögöttes **matematikai alapok** idegenek a mérnöki közönség számára

feladat

[…] A hallgató feladata egy

- **webes eszköz** (*Combinator Explorer*) elkészítése, amely
- SKI-kifejezések esetén megmutatja a **redukciós lépéseket**, vizualizálja a redexeket, és felrajzolja a **kiértékelési fát**.
- A redukció során többféle **kiértékelési stratégiát** támogat, valamint az SKI-kifejezéseket **lefordítja** λ-kalkulusba. […]

— Témakiírás

kezdeti állapot

kezdeti állapot

- Funkcionális programozás C++-ban szabvál $\rightarrow$ **λ-kalkulus**

kezdeti állapot

- Funkcionális programozás C++-ban szabvál → **λ-kalkulus**
- Stephen **Wolfram** írásai → **SKI-kalkulus**

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

$$\lambda x.\lambda y.(x + y)$$

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

$$\lambda x.\lambda y.(x + y)$$

SKI-kalkulus:

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

$$\lambda x.\lambda y.(x + y)$$

SKI-kalkulus:

- Turing-teljes

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

$$\lambda x.\lambda y.(x + y)$$

SKI-kalkulus:

- Turing-teljes
- három kombinátor (S, K, I)

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

$$\lambda x.\lambda y.(x + y)$$

SKI-kalkulus:

- Turing-teljes
- három kombinátor (S, K, I)
- változómentes

λ-kalkulus és SKI-kalkulus

λ-kalkulus:

- általános célú, Turing-teljes nyelv
- változók, absztrakciók, applikációk

$$\lambda x.\lambda y.(x + y)$$

SKI-kalkulus:

- Turing-teljes
- három kombinátor (S, K, I)
- változómentes

$$S = \lambda x.(\lambda y.(\lambda z.xz(yz)))$$
$$K = \lambda x.(\lambda y.x)$$
$$I = \lambda x.x$$

architektúra

implementálási nyelv: **Rust**

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

- UI → `update` → `view` körforgás

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

- UI → `update` → `view` körforgás
- message-queue alapú állapotkezelés

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

- UI → `update` → `view` körforgás
- message-queue alapú állapotkezelés
- tisztán kliensoldali futás (**WASM**)

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

- UI → `update` → `view` körforgás
- message-queue alapú állapotkezelés
- tisztán kliensoldali futás (**WASM**)
- cpu-heavy redukciós számítások kiszervezése **Web Worker**-ekbe

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

backend (compiler felépítés)

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

backend (compiler felépítés)

- compiler frontend: lexer és parser

architektúra

implementálási nyelv: **Rust**

frontend (elm-architektúra)

backend (compiler felépítés)

- compiler frontend: lexer és parser
- compiler backend: G-machine alapú redukciós gépek

architektúra

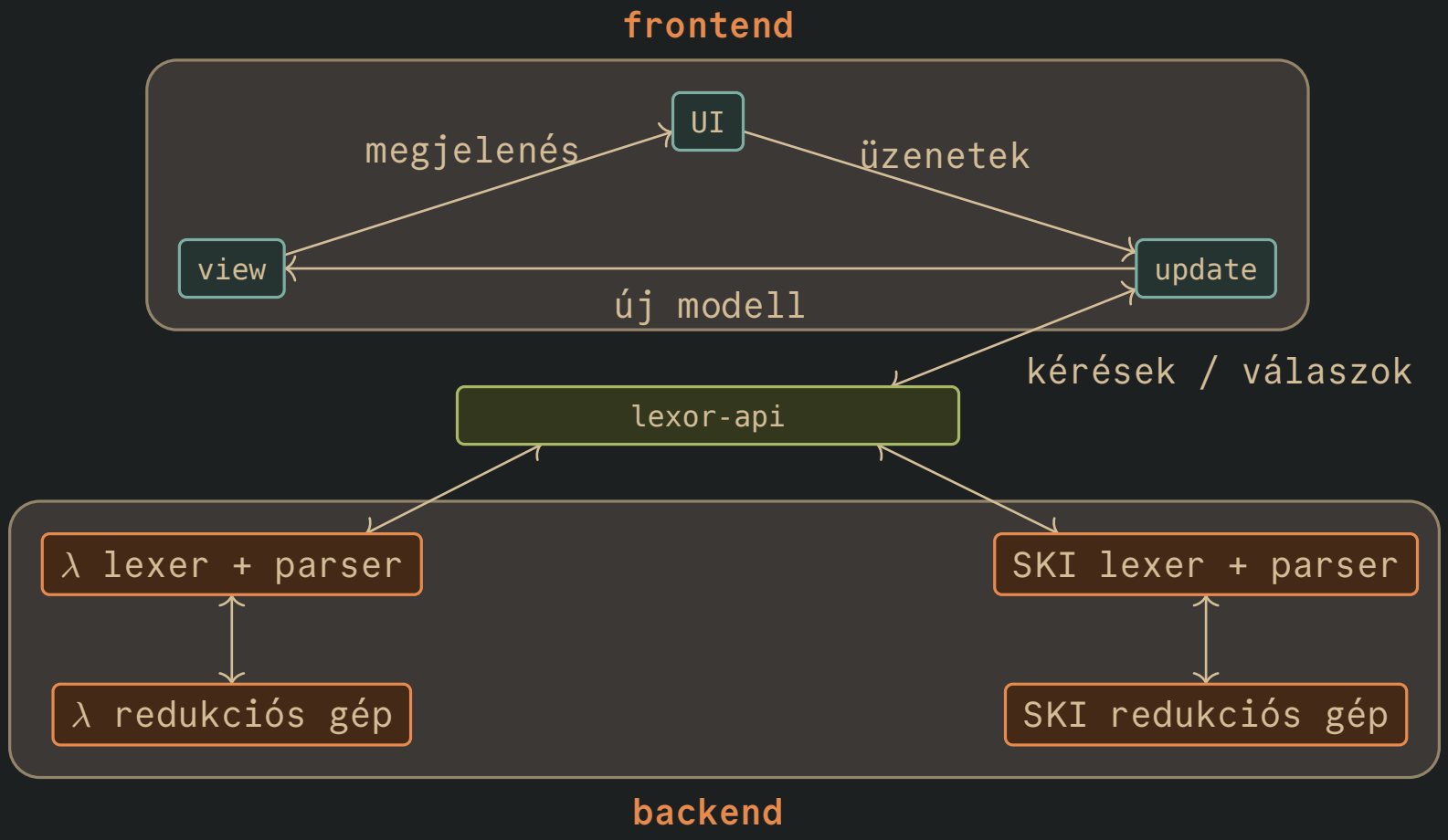
implementálási nyelv: **Rust**

frontend (elm-architektúra)

backend (compiler felépítés)

API réteg: a frontend és backend között **facade** mintával

architektúra



redukciós gépek

graph reduction machine (G-machine)

redukciós gépek

graph reduction machine (G-machine)

tankönyvi algoritmus:

- csúcsok $\rightarrow$ kombinátorok és applikáció

redukciós gépek

graph reduction machine (G-machine)

tankönyvi algoritmus:

- csúcsok $\rightarrow$ kombinátorok és applikáció
- magát az absztrakt szintaxisfát értékeljük ki

redukciós gépek

graph reduction machine (G-machine)

tankönyvi algoritmus:

- csúcsok $\rightarrow$ kombinátorok és applikáció
- magát az absztrakt szintaxisfát értékeljük ki
- közben módosítunk rajta

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus:

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus:

- új féle csúcs: **indirekció**

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus:

- új féle csúcs: **indirekció**
- részfa megosztás

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet:

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet:

- hagyományosan: rekurzív, pointer alapú AST

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet:

- hagyományosan: rekurzív, pointer alapú AST
- pointer chasing! folyamatos allokációk!

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet:

- ~~rekurzív, pointer alapú AST~~
- aréna allokátor alapú Flat-AST

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet

stratégiák:

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet

stratégiák:
- normál sorrend (leftmost-outmost)

redukciós gépek

graph reduction machine (G-machine)

optimalizált algoritmus

adatszerkezet

stratégiák:
- normál sorrend (leftmost-outmost)
- applikatív sorrend (leftmost-innermost)

redukciós gépek

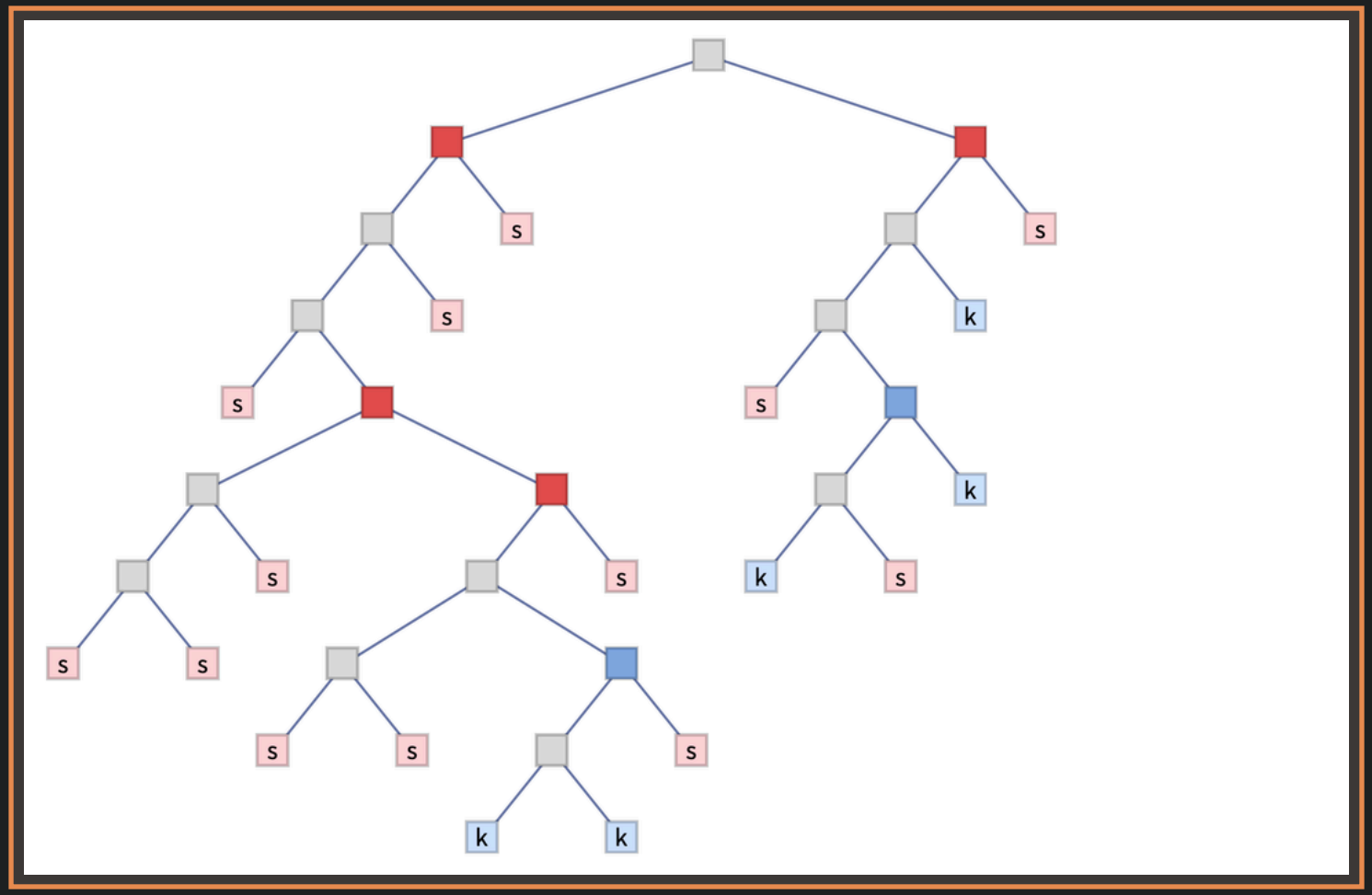
graph reduction machine (G-machine)

optimalizált algoritmus

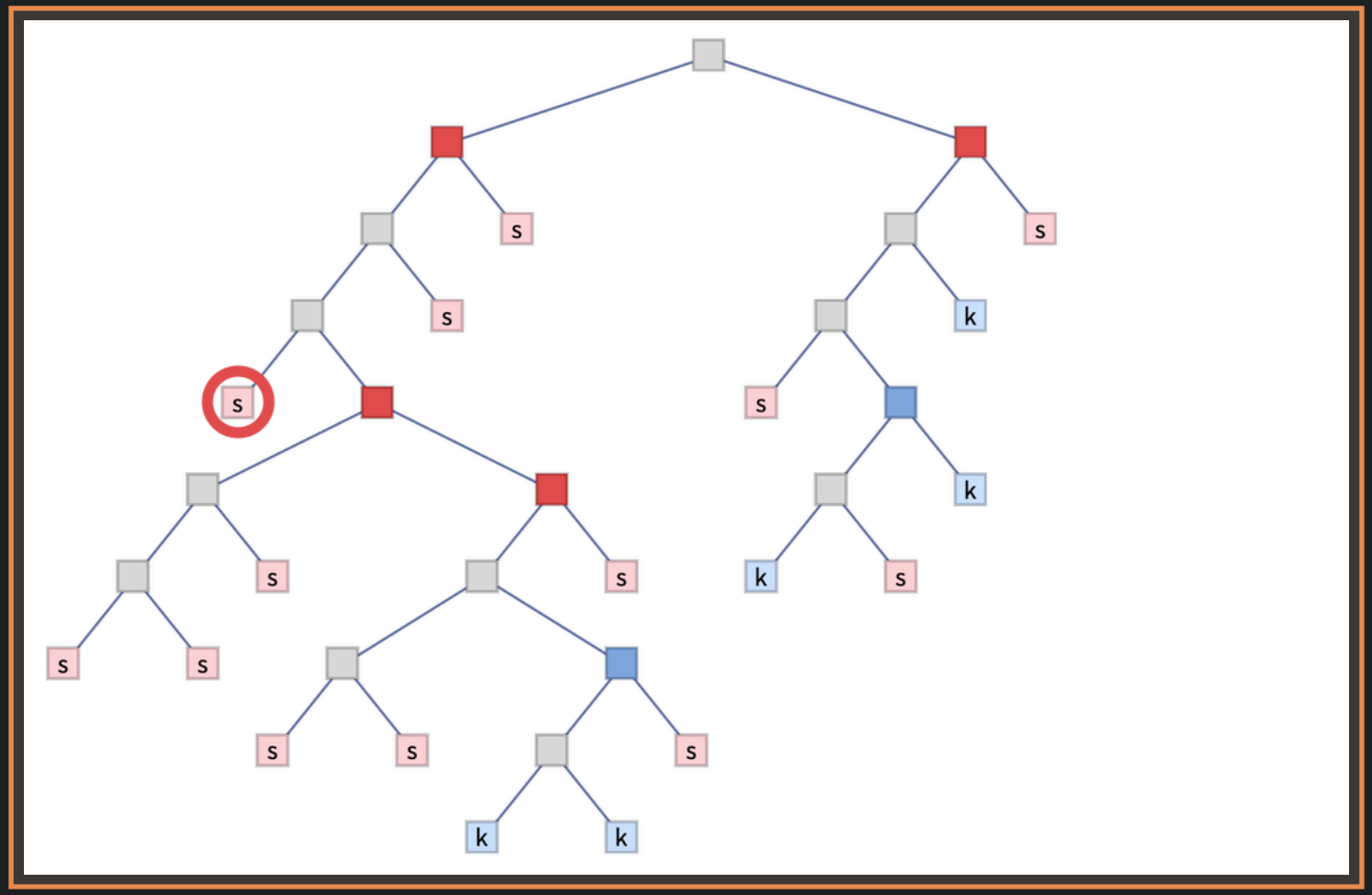
adatszerkezet

stratégiák:
- normál sorrend (leftmost-outmost)
- applikatív sorrend (leftmost-innermost)
- call-by-name/call-by-value (argumentumok kiértékelése)

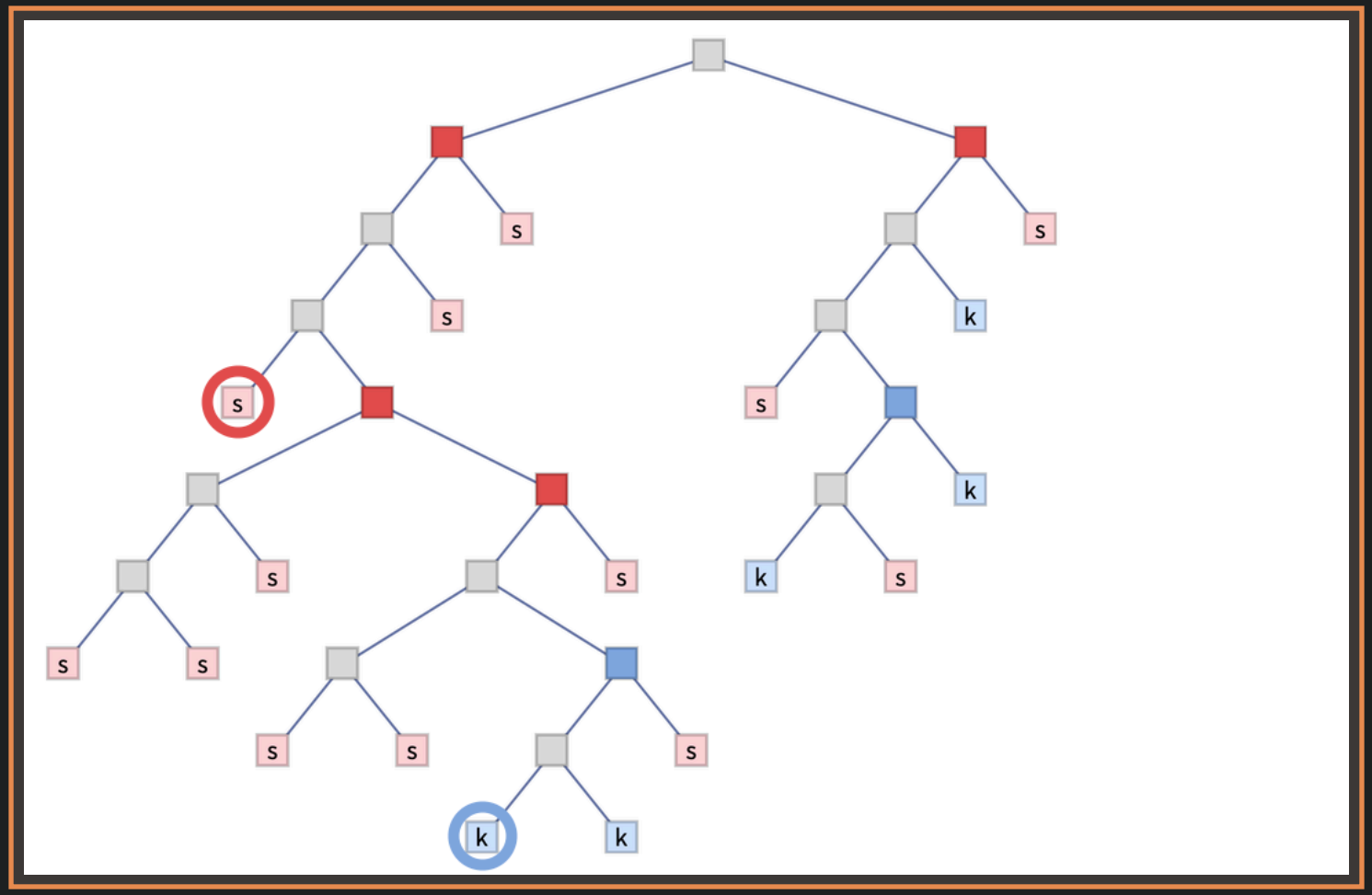
stratégiák



stratégiák (normal order)



stratégiák (applicative order)



λ-kalkulus specifikus kihívások

- a λ-kalkulus kiértékelése bonyolultabb

λ-kalkulus specifikus kihívások

a λ-kalkulus kiértékelése bonyolultabb

- változónevek (α-ekvivalencia)

λ-kalkulus specifikus kihívások

de Bruijn-indexek:

λ-kalkulus specifikus kihívások

de Bruijn-indexek:

- nevek $\rightarrow$ indexek (számok)

λ-kalkulus specifikus kihívások

de Bruijn-indexek:

- nevek $\rightarrow$ indexek (számok)
- hány szinttel feljebb található az őt kötő λ-absztrakció

λ-kalkulus specifikus kihívások

de Bruijn-indexek:

- nevek $\rightarrow$ indexek (számok)
- hány szinttel feljebb található az őt kötő λ-absztrakció

$$\lambda x.\lambda y.x \iff \lambda\lambda 2$$

λ-kalkulus specifikus kihívások

de Bruijn-indexek

oda-vissza konverzió a tradicionális alak és a de Bruijn-forma között

λ-kalkulus specifikus kihívások

de Bruijn-indexek

oda-vissza konverzió a tradicionális alak és a de Bruijn-forma között

(ez segít a SKI $\rightarrow \lambda$ konverzióban is)

tesztelés

egységtesztek mellett **property-based testing**

tesztelés

egységtesztek mellett **property-based testing**

- hasonló a fuzzinghoz

tesztelés

egységtesztek mellett **property-based testing**

orákulum:

tesztelés

egységtesztek mellett **property-based testing**

orákulum:

- külső, helyes interpreter futtatása konténerben

tesztelés

egységtesztek mellett **property-based testing**

orákulum:

- külső, helyes interpreter futtatása konténerben
- eredmények összehasonlítása

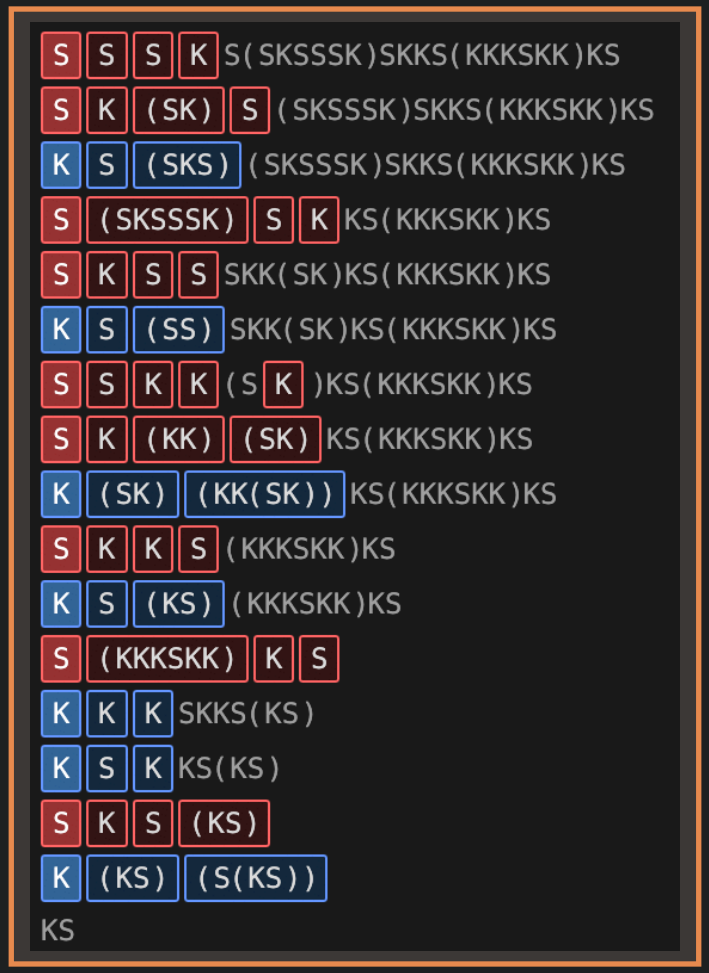
visuals

- a backend JSON-ként továbbítja a redukciós fát

visuals

- a backend JSON-ként továbbítja a redukciós fát
- a frontend **szemantikailag** színezve jeleníti meg a kifejezéseket

visuals


SSSKS(SKSSSK)SKKS(KKKSKK)KS
SK(SK)S(SKSSSK)SKKS(KKKSKK)KS
KS(SKS)(SKSSSK)SKKS(KKKSKK)KS
S(SKSSSK)SKS(KKKSKK)KS
SKSSSKK(SK)KS(KKKSKK)KS
KS(SS)SKK(SK)KS(KKKSKK)KS
SSKK(S K)KS(KKKSKK)KS
SK(KK)(SK)KS(KKKSKK)KS
K(SK)(KK(SK))KS(KKKSKK)KS
SKKS(KKKSKK)KS
KS(KS)(KKKSKK)KS
S(KKKSKK)KS
KKKSKKS(KS)
KSKKS(KS)
SKS(KS)
K(KS)(S(KS))
KS

visuals

- a backend JSON-ként továbbítja a redukciós fát
- a frontend **szemantikailag** színezve jeleníti meg a kifejezéseket

visuals

- a backend JSON-ként továbbítja a redukciós fát
- a frontend **szemantikailag** színezve jeleníti meg a kifejezéseket
- gráf konverzió minden egyes állapothoz

visuals

- a backend JSON-ként továbbítja a redukciós fát
- a frontend **szemantikailag** színezve jeleníti meg a kifejezéseket
- gráf konverzió minden egyes állapothoz
- agresszív cachelés

ui

- egui keretrendszer (Dear ImGui feel)

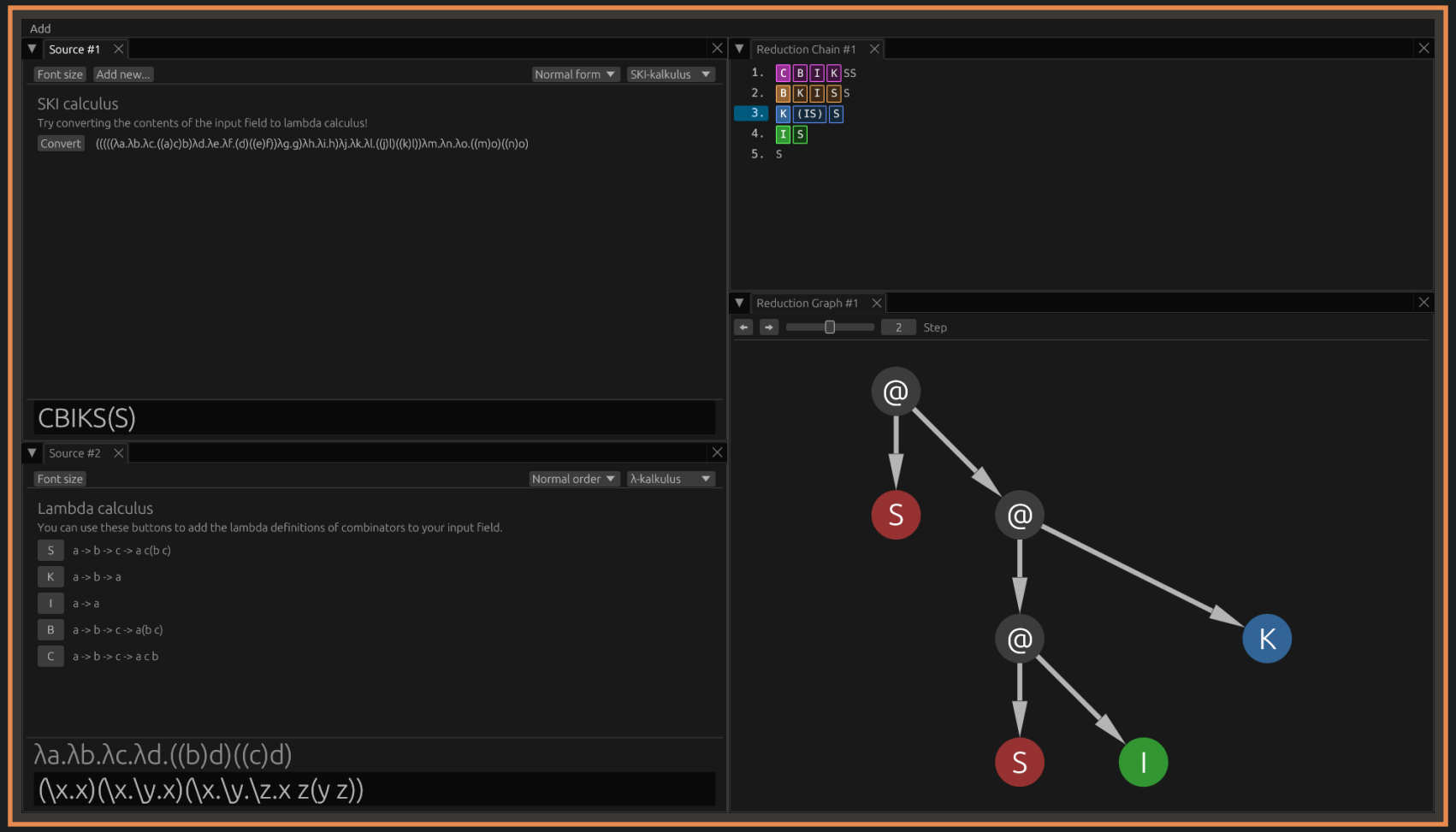
ui

- egui keretrendszer (Dear ImGui feel)
- **immediate-mode** GUI

ui

- egui keretrendszer (Dear ImGui feel)
- **immediate-mode** GUI
- docking support (compiler explorer feel)

ui


Add
Source #1
Font size
Add new...
Normal form
SKI-kalkulus
SKI calculus
Try converting the contents of the input field to lambda calculus!
Convert
(((((λa.λb.λc.((a)c)b)λd.λe.λf.(d)((e)f))λg.g)λh.λi.h)λj.λk.λl.((j)l)((k)l))λm.λn.λo.((m)o)((n)o)
CBIKS(S)
Source #2
Font size
Normal order
λ-kalkulus
Lambda calculus
You can use these buttons to add the lambda definitions of combinators to your input field.
S
a -> b -> c -> a c(b c)
K
a -> b -> a
I
a -> a
B
a -> b -> c -> a(b c)
C
a -> b -> c -> a c b
λa.λb.λc.λd.((b)d)((c)d)
(\x.x)(\x.\y.x)(\x.\y.\z.x z(y z))
Reduction Chain #1
1. C B I K SS
2. B K I S S
3. K (IS) S
4. I S
5. S
Reduction Graph #1
2
Step
@
S
@
@
K
S
I

hova tovább?

hova tovább?

- indirekció ki/be kapcsolása

hova tovább?

- indirekció ki/be kapcsolása
- **guided learning** mód

hova tovább?

- indirekció ki/be kapcsolása
- **guided learning** mód
- nyelvtan bővítése új elemekkel (számok, műveletek)

hova tovább?

- indirekció ki/be kapcsolása
- **guided learning** mód
- nyelvtan bővítése új elemekkel (számok, műveletek)
- $\lambda \rightarrow$ SKI konverzió

hova tovább?

- indirekció ki/be kapcsolása
- **guided learning** mód
- nyelvtan bővítése új elemekkel (számok, műveletek)
- $\lambda \to$ SKI konverzió
- frontend személyre szabhatóságának növelése

összegzés

összegzés

- egy **interaktív** felület a SKI-kalkulussal való ismerkedésre

összegzés

- egy **interaktív** felület a SKI-kalkulussal való ismerkedésre
- gyors redukció

összegzés

- egy **interaktív** felület a SKI-kalkulussal való ismerkedésre
- gyors redukció (mind lépésszám, mind cpu cycles terén)

összegzés

- egy **interaktív** felület a SKI-kalkulussal való ismerkedésre
- gyors redukció (mind lépésszám, mind cpu cycles terén)
- intuitív vizuális segítő elemek

Köszönöm a figyelmet!